

Git × Trae × AI 协同

抖音 AI 创变者计划 2026 · 广州站

这份材料讲什么

1. **Git 的基本概念**：文件状态、三个区、常用命令
2. **分支与合并**：怎么不互相覆盖、怎么解决冲突
3. **GitHub 协作**：clone、fork、Pull Request 的完整流程
4. **Trae 编辑器**：AI 在编辑器里直接调用工具
5. **用 AI 操作 Git**：用对话替代命令，以及风险边界
6. **比赛实战清单**：赛前、赛中、出问题时的操作步骤

Part 1: 为什么要懂 Git

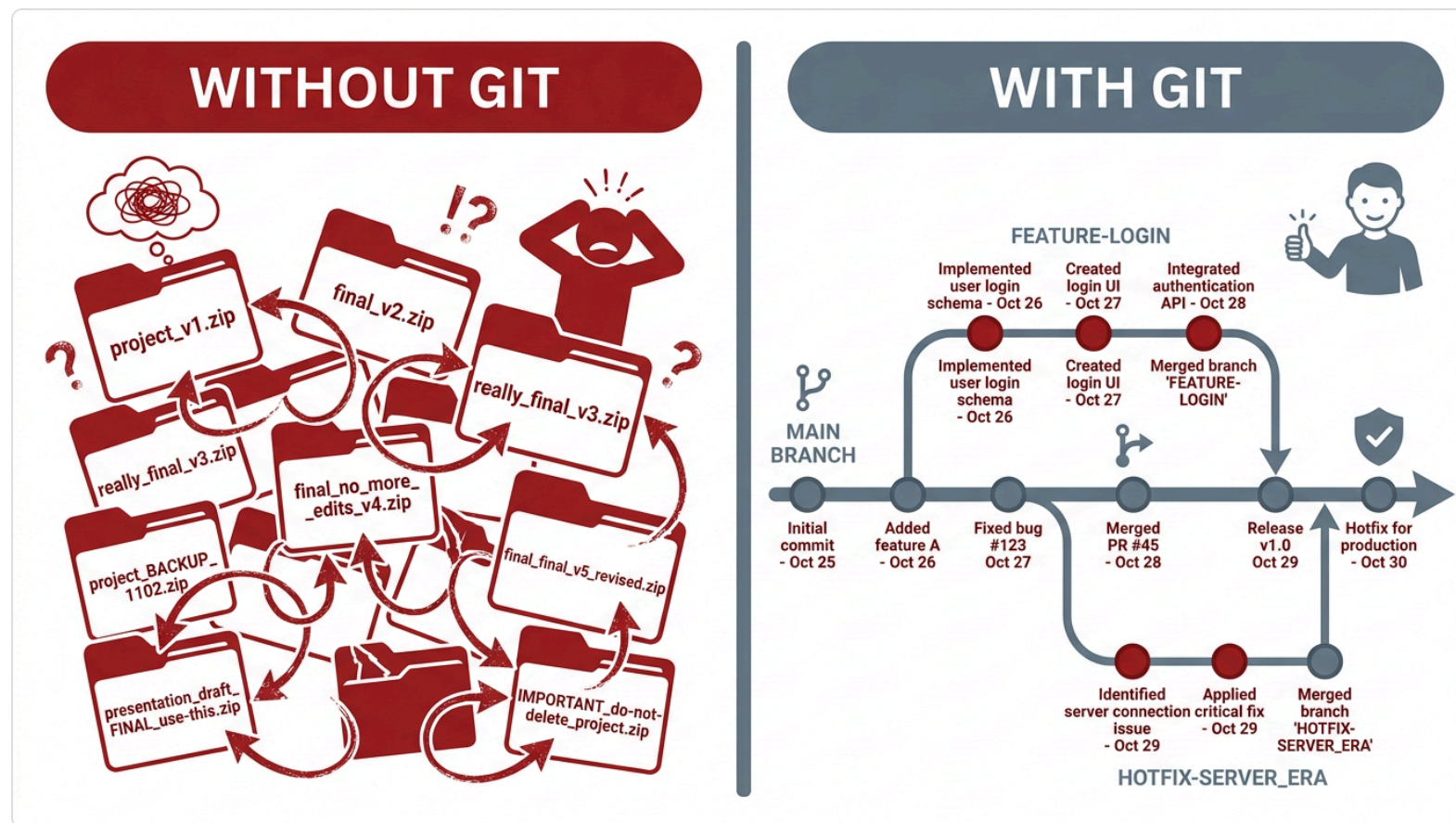
不用 Git 协作时的常见问题

没有版本控制时，团队几乎一定会遇到下面四种情况：

- **互相覆盖**：A 改完发到群里，B 还在用旧版本改自己的，最后 B 一覆盖，A 的工作没了
- **版本命名失控**：文件夹里堆着 `项目_v1.zip`、`最终版.zip`、`真的最终版_v3.zip`，没人知道哪份是当前进度
- **不敢动别人写的代码**：怕改坏，最后所有人只敢碰自己那一小块
- **合并靠肉眼**：临交付前把几个人的代码人工拼起来，常常通宵还出 bug

这些问题的共同根源： 项目没有一份"当前真实状态"的权威记录。

用与不用 Git 的差别



左侧：每人本地的文件夹各自演化，状态不可追溯。右侧：Git 把所有人的修改记录在同一条时间线，可以随时回看、对比、合并。

Git 是什么——时光机

Git 是一个**记录文件变更历史**的工具。它做三件事：

- **保存快照**：每次提交都保存一份完整的项目状态，可以回到任意一次提交时的样子
- **支持并行开发**：每个人可以在自己的分支上独立修改，互不影响，再合并到一起
- **可追溯**：每一行代码都能查到是谁、什么时候、为什么改的（用 `git blame`）

Git 是本地工具，**GitHub / Gitee 是托管 Git 仓库的网站**——两者不是一回事。
本地用 Git 命令操作，GitHub 提供云端备份、协作和代码审查界面。

Part 2: Git 的基本概念

文件在 Git 里的四种状态

任何一个文件在 Git 仓库里，处于以下四种状态之一：

- **未跟踪 (Untracked)**：新建的文件，Git 还不管它
- **已修改 (Modified)**：跟踪中的文件被改了，但还没决定是否提交
- **已暂存 (Staged)**：你已经"挑出来"准备打包进下一次提交的改动
- **已提交 (Committed)**：已经永久写入版本库的快照

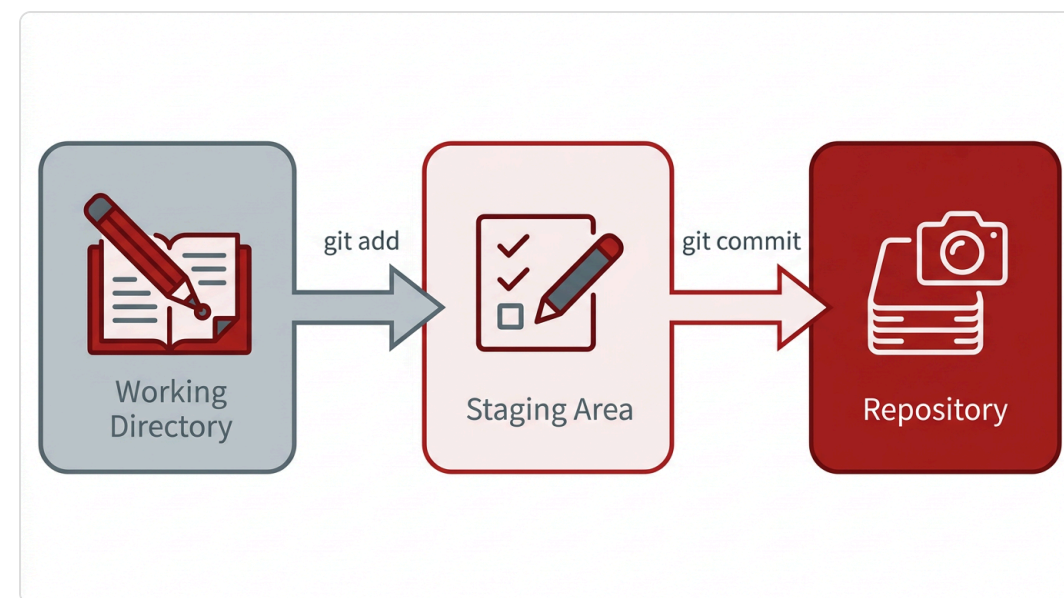
`git status` 会告诉你每个文件当前在哪个状态。看不懂状态时，先跑 `git status`，它会列出每一类文件并给出下一步建议命令。

三个区：状态在哪里流转

文件状态的流转对应三个"区"：

- **工作区**：你正在编辑的文件（未跟踪 / 已修改）
- **暂存区**：用 `git add` 挑进来的改动（已暂存）
- **版本库**：用 `git commit` 写入的历史（已提交）

暂存区的设计让你可以**只挑一部分改动来提交**，而不是把当前所有修改一股脑打包。



日常最常用的命令（按用途分组）

基本节奏是从上到下：看状态 → 挑改动 → 提交 → 同步到远程。

用途	命令	说明
看状态	<code>git status</code>	哪些文件改了、在哪个状态
加入暂存	<code>git add <文件></code>	挑出要打包的改动
提交	<code>git commit -m "说明"</code>	生成一次快照
看历史	<code>git log --oneline</code>	列出所有提交
看差异	<code>git diff</code>	工作区相比上次提交改了什么
同步远程	<code>git pull</code> / <code>git push</code>	从远程拉 / 推到远程

一次完整的提交步骤

1. 看哪些文件改了

```
git status
```

2. 挑出要提交的文件（可以一次加多个）

```
git add src/Login.tsx src/api/auth.ts
```

3. 写一句说明，生成提交

```
git commit -m "feat: 实现登录页面与接口对接"
```

4. 推到远程（同步给队友）

```
git push
```

每完成一个**小而完整的功能**就走一次这个循环。不要等写完整个项目才提交，那样出问题无法回滚到中间状态。

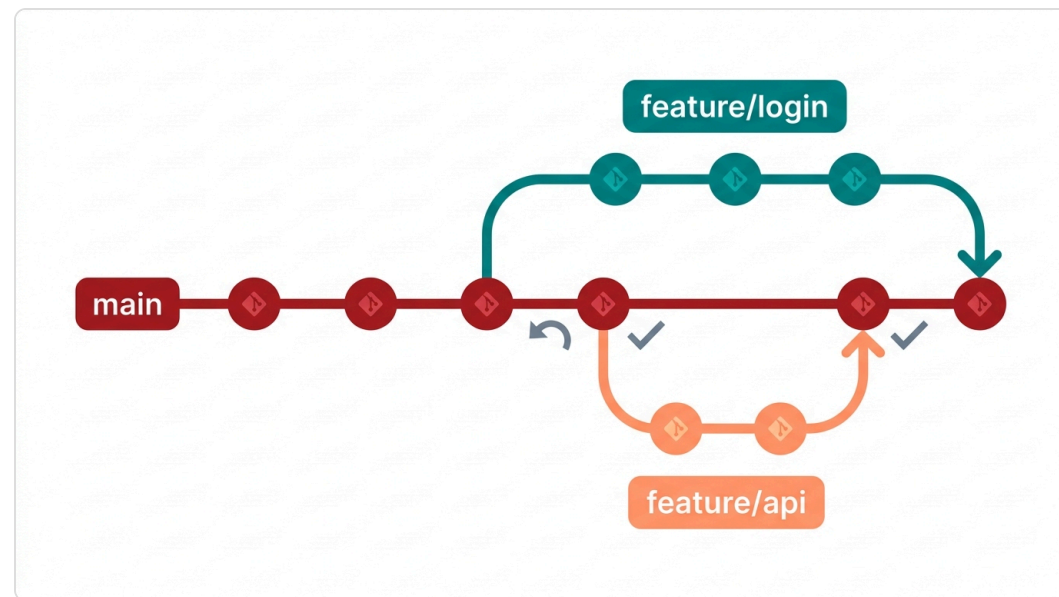
看历史与撤销操作——建立初步印象就可以

场景	命令	说明
看最近 10 次提交	<code>git log --oneline -10</code>	标题列表，快速找节点
看某文件的修改历史	<code>git log <文件></code>	这个文件被谁、何时改过
看工作区改了什么	<code>git diff</code>	还没 add 的改动
撤销工作区改动	<code>git restore <文件></code>	恢复到上次提交的样子（ 改动会丢 ）
把暂存区的撤回	<code>git restore --staged <文件></code>	撤回 add，改动还在工作区
撤销最近一次提交	<code>git reset --soft HEAD~1</code>	提交没了，改动还在暂存区
撤销并丢弃改动	<code>git reset --hard HEAD~1</code>	不可逆，慎用

分支：独立的开发线

分支让多人**同时改项目**而不互相影响。

- `main`：主线，要求始终可运行，是 demo 用的版本
- `feature/xxx`：你为某个功能临时开的分支，写完合并回 `main` 后可以删掉
- 切到新分支：`git checkout -b feature/xxx`
- 切回主线：`git checkout main`
- 看本地有哪些分支：`git branch`



一次完整的协作流程

主线之所以重要：队友任何时候都能从 main 拉一份能跑的代码。

```
# 1. 从 main 建一个新分支，名字写清楚要做什么
git checkout -b feature/login-page

# 2. 写代码 ... 写完看动了哪些文件
git status

# 3. 挑选要提交的文件，写说明
git add src/Login.tsx
git commit -m "feat: 实现登录页面基础布局"

# 4. 推到远程（第一次推某分支要加 -u origin <分支名>）
git push -u origin feature/login-page

# 5. 去 GitHub / Gitee 网页发起 Pull Request，请队友 review
# 6. 队友通过后，在网页上点 Merge，分支合并回 main
```

Part 3: 用 GitHub 协作

GitHub 是什么

GitHub 是托管 Git 仓库的网站（国内类似的有 Gitee）。它在 Git 之上加了：

- **远程仓库**：把代码备份到云端，所有队员都能拉取和推送
- **Pull Request**：发起代码合并请求，队友可以逐行评论后再合并
- **Issue**：记录任务、bug 和讨论
- **权限管理**：决定谁能 push、谁能合并到 main

注册账号后第一件事：生成 SSH key 或 Personal Access Token，否则 push 时会反复要密码。具体步骤可以让 AI 直接告诉你如何操作。

拉取一个仓库到本地：clone

`git clone` 把远程仓库完整复制到本地，包括所有历史和分支。

```
# HTTPS 方式（任何网络都能用，每次 push 要输 token）
git clone https://github.com/your-team/project.git

# SSH 方式（配过 key 后免密，推荐）
git clone git@github.com:your-team/project.git

# clone 后进入项目目录
cd project
```

clone 一次就够了，之后用 `git pull` 拉队友的最新提交，不需要重新 clone。**clone 是把"完整时间线"搬到本地**，不是只下载当前项目文件。

clone 和 fork 的区别

操作	在哪里发生	用于什么场景
clone	把远程仓库下载到本地电脑	队员拉自己队伍仓库；下载别人的开源代码看
fork	在 GitHub 上把别人的仓库复制一份到自己账号	想给开源项目贡献代码，但你没有原仓库的 push 权限

比赛中的典型用法：

- 拉自家队伍仓库 → 直接 **clone**
- 想参考开源项目做自己的魔改版 → **fork** 到自己账号，再 clone 下来改
- 想给开源项目修 bug 提交回去 → fork、clone、改、push，最后在原仓库发起 PR

Pull Request 的完整流程

PR 是 GitHub 上的代码审查机制。从写完代码到合并入 main 的标准步骤：

1. **本地写代码**：在 `feature/xxx` 分支上完成功能，commit 后 push 到远程
2. **GitHub 上发起 PR**：选择从 `feature/xxx` 合并到 `main`，填标题和说明（改了什么、为什么、怎么测）
3. **队友 review**：在文件 diff 上逐行评论，提出修改建议
4. **你回应评论**：要么改代码再 push（PR 会自动更新），要么解释为什么这样写
5. **通过后合并**：在 GitHub 页面点 Merge，分支合并进 main
6. **清理**：删掉远程和本地的 feature 分支，本地 `git pull` 拉最新 main

PR 不是流程，是沟通工具——比起在群里贴代码，PR 让讨论沉淀在 diff 旁边。

合并冲突：原因与解决

触发条件： 你和队友改了**同一文件的同一行**，Git 不知道该留谁的。

冲突发生时，Git 会在文件里插入下面这种标记：

```
<<<<<<< HEAD
你的版本
=====
队友的版本
>>>>>>> feature/xxx
```

解决步骤： 打开文件，决定保留哪一边（多数时候要把两边整合），删掉 `<<<<<<<` / `=====` / `>>>>>>>` 三行标记，然后 `git add <文件>` + `git commit` 完成合并。

解决冲突时的三条原则

- **先看清各自改了什么：**不要急着删，先理解两边的意图
- **多数时候要合并双方：**不是二选一，而是把两个改动整合在一起
- **不确定就找队友确认：**30 秒沟通胜过默默改错半小时

解决完冲突，**先在本地把代码跑一遍**，确认没坏再 commit。冲突解决错了，往往在合并后才暴露。

Part 4: Trae 是什么

Trae 是什么

Trae 是一个内置 AI 的代码编辑器，界面和 VSCode 基本一致。

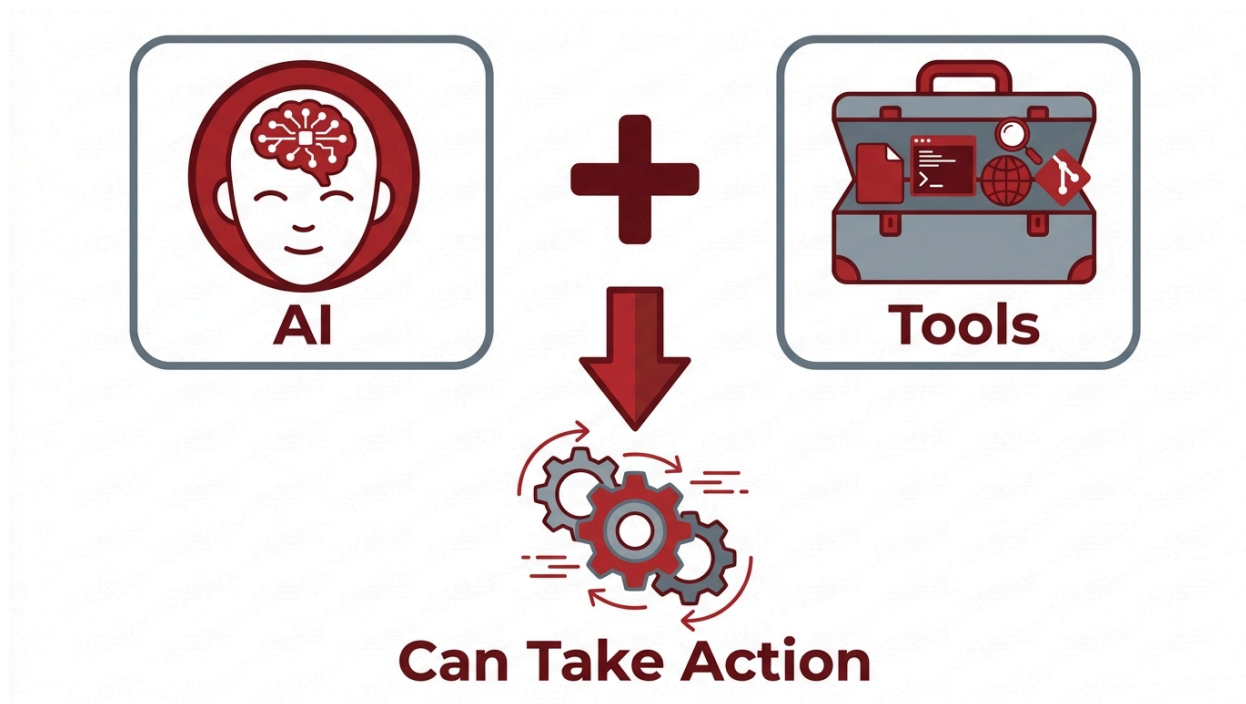
- **看得到你的项目：**AI 对话窗口可以读取当前打开的文件、整个项目结构
- **能调用工具：**不仅仅是给建议，还能直接读文件、改文件、在终端跑命令
- **保留审阅权：**每一步动作都会展示给你看，由你点"允许"或"拒绝"

不需要在浏览器和编辑器之间反复切换，AI 在你的项目里直接工作。



Agent = AI + 工具

只能对话的 AI 只能给建议；接入工具后，AI 才能完成“读懂当前项目—修改文件—运行验证”这样的完整任务。Trae 里的 AI 属于后者。



AI 在 Trae 里能做的事

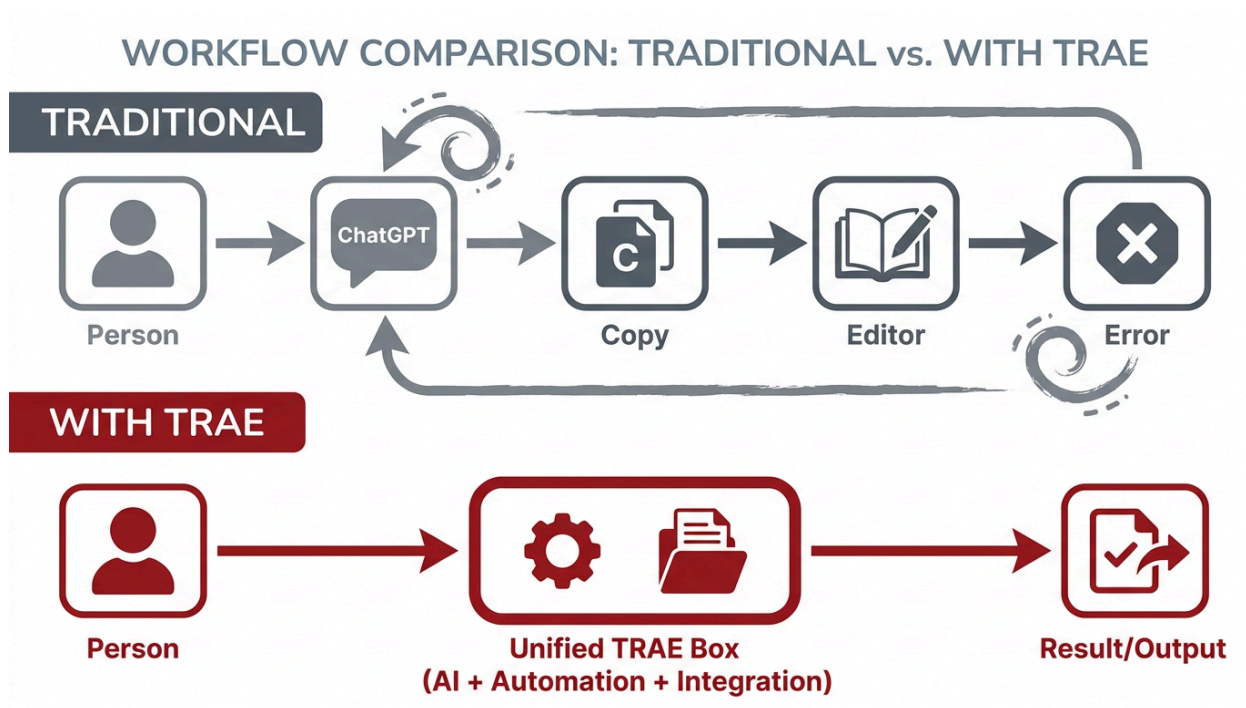
启动一次对话后，AI 通常会用到下面这些工具：

- **读文件**：理解项目结构、找到相关代码位置
- **改文件**：在指定文件、指定位置直接写代码，不需要你复制粘贴
- **跑命令**：在终端执行 `npm install`、`pytest`、`git commit` 等
- **搜索**：在整个项目里查找符号定义、引用、关键字
- **联网查资料**：访问网页查官方文档或最新 API

关键约束： 每个动作（尤其是改文件和跑命令）执行前都会弹出确认，由你点"允许"或"拒绝"。

与传统工作流的对比

传统流程，你在浏览器（ChatGPT）和编辑器之间反复复制粘贴。
在 Trae 里，AI 直接在项目里读写文件、跑命令，你只需要确认和指导方向。



Part 5: 用 AI 操作 Git

为什么用 AI 来操作 Git

Git 命令本身的设计有几个对新手不友好的地方：

- 相似命令做的事完全不同：`git reset --hard` 会丢失改动，`git reset --soft` 只是撤销提交
- 报错信息往往描述现象不解释原因：例如 `rejected non-fast-forward`，看名字不知道该怎么办
- 危险操作没有二次确认：一行命令就能覆盖远程或抹掉本地未提交的工作
- 合并冲突的标记格式陌生，第一次见的人不知道在哪里改

用 AI 操作的好处： 你用自然语言描述意图，AI 把它翻译成正确的命令序列，**执行前先展示给你看**，确认无误再跑。

按风险把指令分四类

类型	风险	例子
查询类	低	"看一下现在哪些文件改了"
提交类	中	"把登录页相关改动提交，message 你写"
协作类	高	"拉队友最新代码，告诉我冲突在哪"
撤销类	极高	"刚才的 commit 写错了，帮我撤销"

风险越高，越要让 AI 先解释方案、确认后再执行。撤销类指令最好让 AI 列出 1-2 种方案对比给你看，由你选。

写好提示词的四个要点

- **说目标，不说命令**：让 AI 自己挑合适的命令。你不需要记住 `reset --soft` 还是 `restore --staged`。
- **给上下文**：当前在哪个分支、有没有未提交的改动、目标是推到哪里。上下文越具体，AI 越不容易猜错。
- **要求先解释再执行**：在提示词里直接写"先告诉我你打算跑什么命令"，拿到方案确认后再点允许。
- **危险操作双重确认**：涉及强推、删分支、`reset --hard` 时，要求 AI 先讲清后果。

写好提示词的四个要点

反例 vs 正例：

"跑 `git reset --hard HEAD~1`"



"刚才那个 commit 写错了，帮我撤销，但本地改动要留着"

- 注意可以让 AI 不直接执行——它先列出计划，等你点允许。

一个完整对话示例

你：我在 `feature/login` 写完登录页，帮我提交并推到远程，commit message 按 conventional commits 规范写。

AI：好，我先看改了哪些文件。

(执行 `git status`，列出 3 个改动文件)

我打算执行下面三条命令：

```
git add src/Login.tsx src/api/auth.ts src/styles/login.css
git commit -m "feat(login): 实现登录页与接口对接"
git push origin feature/login
```

确认执行吗？

你：执行。

让 AI 写 commit message

commit message 是对本次提交的操作的解释，这是给过后自己和其他队友看的。AI 比人有耐心，每次都能写非常完整。

提示词模板：

"看一下我暂存区的改动，按 conventional commits 规范写一条 commit message。标题 50 字以内说明做了什么，body 写为什么这么改。"

典型输出：

```
feat(auth): 增加 JWT 刷新机制
```

避免用户在长会话中被强制登出。refresh token 有效期 7 天，access token 缩短到 15 分钟，刷新逻辑放在 axios 拦截器里。

让 AI 解决合并冲突

冲突标记初学者不熟悉，AI 读起来没有障碍。

提示词模板：

| "哪些文件有冲突？挑第一个，解释两边各想做什么、建议保留哪边、为什么。"

AI 会：

- 读冲突文件，对比 HEAD 和合入分支两边的版本
- 解释两边各自的修改意图，而不是机械二选一
- 给出合并建议（通常是整合两边），你确认后再写回文件

关键原则： 让 AI **解释和建议**，最终决定权在你。冲突解错了往往要花数倍时间排查。

不建议让 AI 直接执行的操作

下面这些操作必须让 AI 先解释清楚后果，由你亲自决定：

- `git push --force`：会覆盖远程历史，可能抹掉队友刚 push 上去的提交
- `git reset --hard`：本地工作区和暂存区的未提交改动会全部消失，且无法恢复
- 对已推送分支用 `git rebase`：重写历史后，队友再 pull 时会冲突或错乱
- `git branch -D <分支>`：强制删除分支，如果有未合并的工作会丢失
- 提交 `.env` / `secrets.json` 等敏感文件：一旦推到远程，即使删除历史也可能已经被爬走

AI 写代码不等于你学会代码

比赛只有2天，但之后与AI协作的时间还很长。

- 让 AI 写完代码后，**花 30 秒读一遍它写了什么**，至少知道大致逻辑
- 看不懂的地方追问："这段为什么这么写？有没有更好更稳定的实现？"
- 把 AI 当作"会随时讲解的老工程师"，而不是"代码自动贩卖机"

比赛能带走的不只是仓库里的代码，更是建立起来的、对工具和工作流的理解。

Part 6: 比赛实战清单

赛前环境就位

队长要做的：

1. 在 GitHub 或 Gitee 建仓库，把所有队员加为协作者
2. 推一个初始 README 和 `.gitignore`（让大家有同一个起点）
3. 在仓库设置里**保护 main 分支**：勾选 "Require pull request before merging"，避免有人直接推到主线

每位队员要做的：

1. 安装 Trae 并登录账号
2. `git clone <仓库地址>` 把仓库拉到本地
3. 配置身份：`git config user.name "你的真名"` 和 `git config user.email`

每写一个小功能的标准流程

动手前

- git pull main
- 建新分支 feature/xxx
- 跟队友说一声你准备做什么

开发中

- 写完一小段/一个功能就 commit
- commit message 写清楚
- 写完就 push 备份

完成后

- 本地跑通再发 PR
- PR 描述写清楚改了什么 (AI辅助)
- 等一个队友 review

合并后

- 删本地旧分支
- pull 最新 main
- 开始下一个小功能

常见问题与应对方法

出现的情况	给 AI 的指令
改了一堆代码才发现忘记建分支	"我在 main 上改了一堆没提交的东西，帮我搬到新分支 feature/xxx"
提交了不该提交的大文件（如数据集）	"上一个 commit 把 dataset.zip 提交了，帮我从这次提交里移除"
pull 之后冲突看不懂	"现在哪些文件有冲突？挑一个解释冲在哪、建议怎么留"
不小心 push 到了错误分支	"我把 dev 的代码推到 main 了，帮我安全地恢复 main"
push 报错 rejected non-fast-forward	"把报错原文给你，先解释为什么这样，再给我安全的解决步骤"

比赛中的三条核心原则

- **每完成一个小功能就 push 一次**：比赛只有两天，节奏快。死机、误删、改坏都可能瞬间发生，push 到远程是最便宜的保险。不要积攒几小时的改动才同步。
- **PR 尽量小**：一次 PR 只解决一件事，几十行最佳。队友 5 分钟能 review 完，问题也容易定位。功能复杂就拆成多个连续 PR。
- **commit message 写清楚改了什么**：临时一句"修复一下"等于没写。哪怕只有一行改动，也写清是改了什么、为什么。出问题回滚时这是唯一的线索。

赛末两小时不要再合大的 PR——留时间给前后端联调、demo 演练和 bug 修复。

团队协作的几条约定

- **改公共文件前在群里说一声**：减少冲突最简单有效的办法
- **PR 自己先 review 一遍**：发出去之前过一眼自己的 diff，能省队友一半时间
- **review 别人的代码先肯定再提建议**：评审是协作，不是评判
- **合并冲突由"后改的人"解决**：先 push 的人不应该被反复打扰
- **接手别人的分支前先 pull 最新版**：不要在过时的基础上开始新工作

通用心态：默认队友很累、信息不全；多一句解释，少一次返工。

总结

Git: 文件四种状态、三个区、几组常用命令、分支与合并。

GitHub: clone 拉代码、fork 复制别人的仓库、PR 走代码审查。

Trae + AI: 用自然语言指挥 AI, AI 调用工具完成操作; 危险操作始终由你确认。

谢谢！

祝各位赛场顺利

抖音 AI 创变者计划 2026 · 广州站